

MCP y casos de uso

MMIA 6013 — IA Generativa y Agentes · Sesión 13 · miércoles 30 de septiembre de 2026

Semana 3 · RA-4 · 78 minutos de deck y discusión, 12 de la cuenta en grupos, 20 de cuaderno

Tercera sesión de la semana de agentes. El Taller 3 es el sábado 3 de octubre

Daniel Andrés Riofrío Almeida

Ayer, qué herramienta escribir. Hoy, cómo se descubre

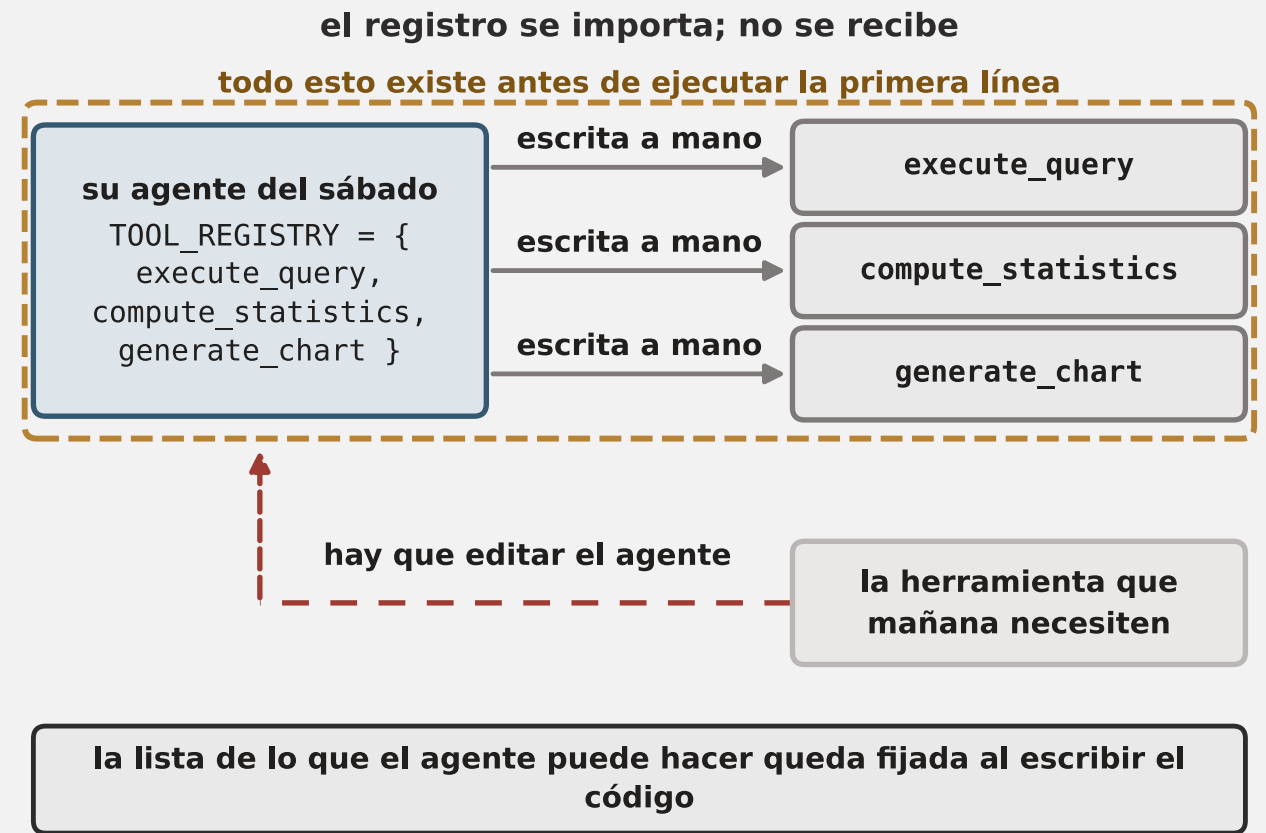
- Cerré ayer con esa frase, y es literalmente el temario de hoy: solo eso cambia
- El lunes fijamos el bucle y la traza; ayer, el patrón y el contrato de la herramienta
- Nada de eso se redefine hoy. No se vuelve a explicar function calling ni el esquema
- Lo único que cambia es de dónde sale la lista de lo que el modelo puede pedir
- Primera mitad: la cuenta que justifica un protocolo, y cuándo NO lo justifica
- Segunda mitad: qué es MCP, las tres cosas que no es, y los tres casos de uso del sílabo

La cuenta que justifica un protocolo

13

Su agente conoce sus herramientas en tiempo de importación

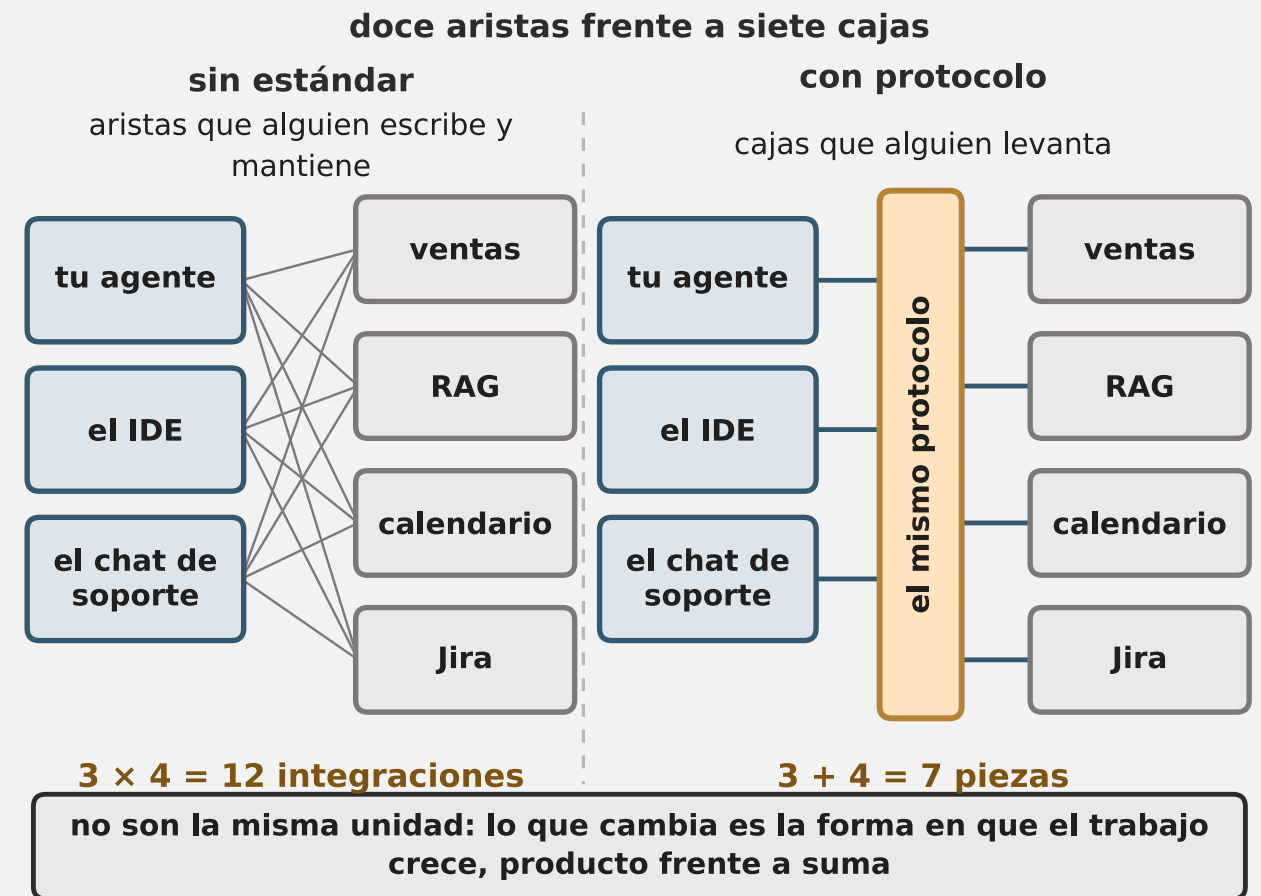
- El cuaderno del lunes: TOOLS es un diccionario literal de tres nombres, escrito a mano
- El andamiaje que califico el sábado: TOOL_REGISTRY, otro diccionario de tres nombres
- agent.py hace TOOL_REGISTRY.get(name), y ese registro se IMPORTA, no se recibe
- Consecuencia: la lista de lo que el agente puede hacer queda fijada al escribir el código
- Ésa es la propiedad —y la única— que hoy se rompe
- La lectura previa pedía traer respondido: de las cinco formas de conectar, ¿en cuál está su Lab 03?



los 3 nombres se leen de tools.py; son las 3 herramientas obligatorias del Taller 3

M×N cuenta integraciones; M+N cuenta piezas

- M aplicaciones —su agente, un IDE, el chat de soporte, un cron— y N herramientas
- Sin estándar: cada par aplicación–herramienta es una integración que alguien escribe y mantiene
- Con protocolo: cada herramienta se expone UNA vez como servidor; cada aplicación habla como cliente
- M×N cuenta integraciones. M+N cuenta piezas: N servidores más M clientes
- No son la misma unidad. La comparación NO es un factor de mejora de rendimiento
- Lo que cambia es la forma en que el trabajo crece: producto frente a suma



la banda central no es una pieza que usted escriba

Con los números de su Lab 03, ¿la cuenta les da la razón?

$M \cdot N > M + N \iff M \cdot N - M - N + 1 > 1$
 $1 > 1 \iff (M - 1)(N - 1) > 1$

$M = 1, N = 3 \rightarrow$ cableado 3, protocolo 4.

$M = 1, N = 10 \rightarrow$ 10 frente a 11

$M = 2, N = 2 \rightarrow$ empate en 4. $M = 2, N = 3 \rightarrow$ 6 frente a 5. $M = 5, N = 5 \rightarrow$ 25 frente a 10

Con UN solo cliente el protocolo nunca compensa, sea cual sea el número de herramientas

Su Lab 03 es exactamente $M = 1$ y $N = 3$: un agente, tres herramientas obligatorias

(a) ¿La cuenta les da la razón? (b) Entonces, ¿por qué harían la extensión B?

cuándo $M \cdot N$ supera a $M + N$

$M \cdot N > M + N \iff M \cdot N - M - N + 1 > 1 \iff (M - 1)(N - 1) > 1$

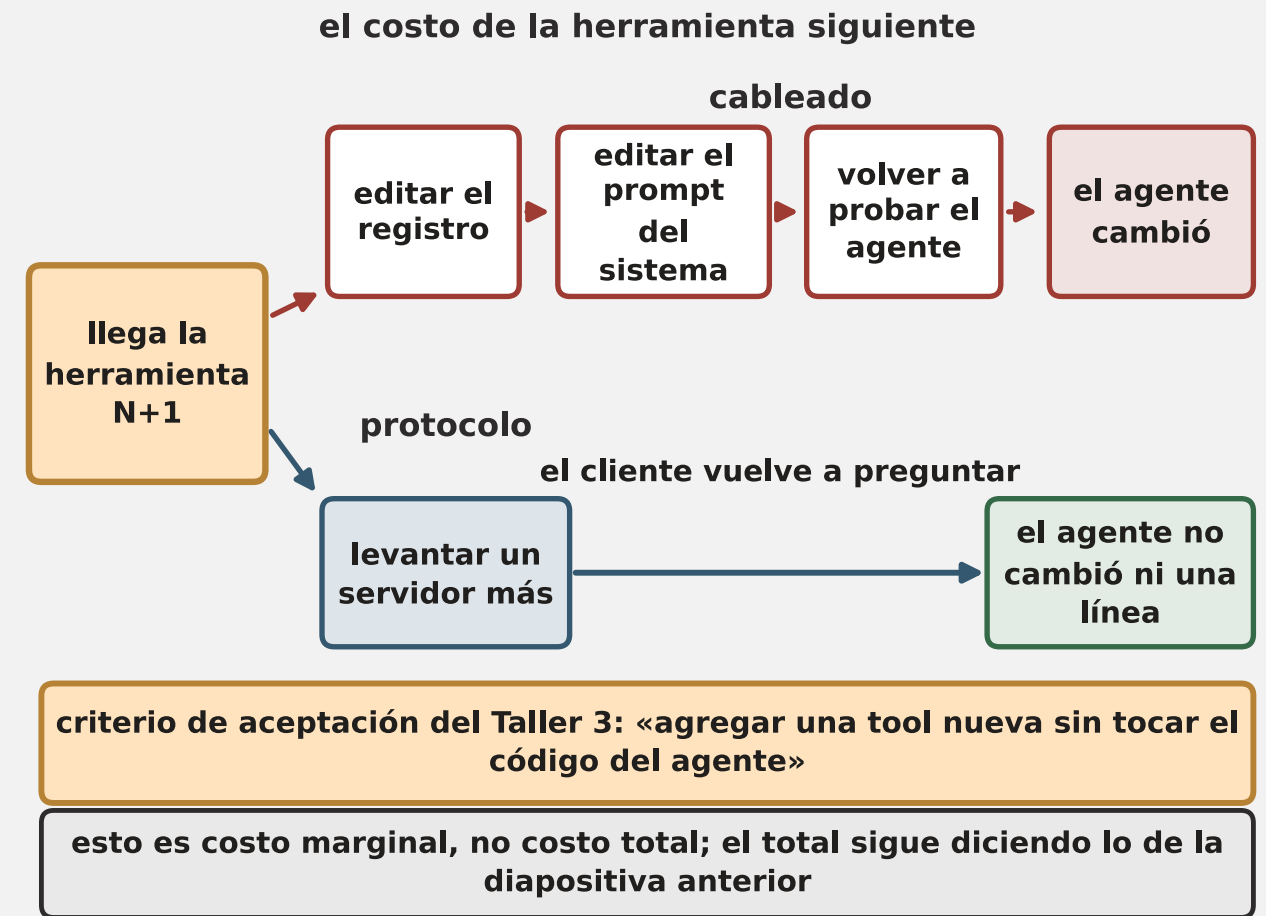
	M	N	cableado	protocolo
	aplicaciones	herramientas	$M \cdot N$	$M + N$
su Lab 03	1	3	3	4 (+1)
	1	10	10	11 (+1)
	2	2	4	4 (=)
	2	3	6	5 (-1)
	5	5	25	10 (-15)

$M \cdot N$ cuenta integraciones; $M + N$ cuenta piezas: no son la misma unidad

aritmética; ninguna cifra depende de la adopción del protocolo

Con un solo cliente el argumento no es el total: es la herramienta N+1

- Con cableado, cada herramienta nueva edita el agente: el registro, el prompt y las pruebas
- Con protocolo, se levanta un servidor más y el agente no cambia ni una línea
- El taller lo convierte en criterio de aceptación, literal: «agregar una tool nueva sin tocar el código del agente»
- Segundo argumento: N puede crecer sin que ustedes escriban los servidores
- Es el criterio del lunes aplicado al protocolo: prefiera lo simple mientras alcance
- Un protocolo se justifica por la estructura del problema, no por ser lo más nuevo



el cuaderno de hoy lo demuestra levantando un segundo servidor

El mecanismo, y con qué respaldo lo cuento

13

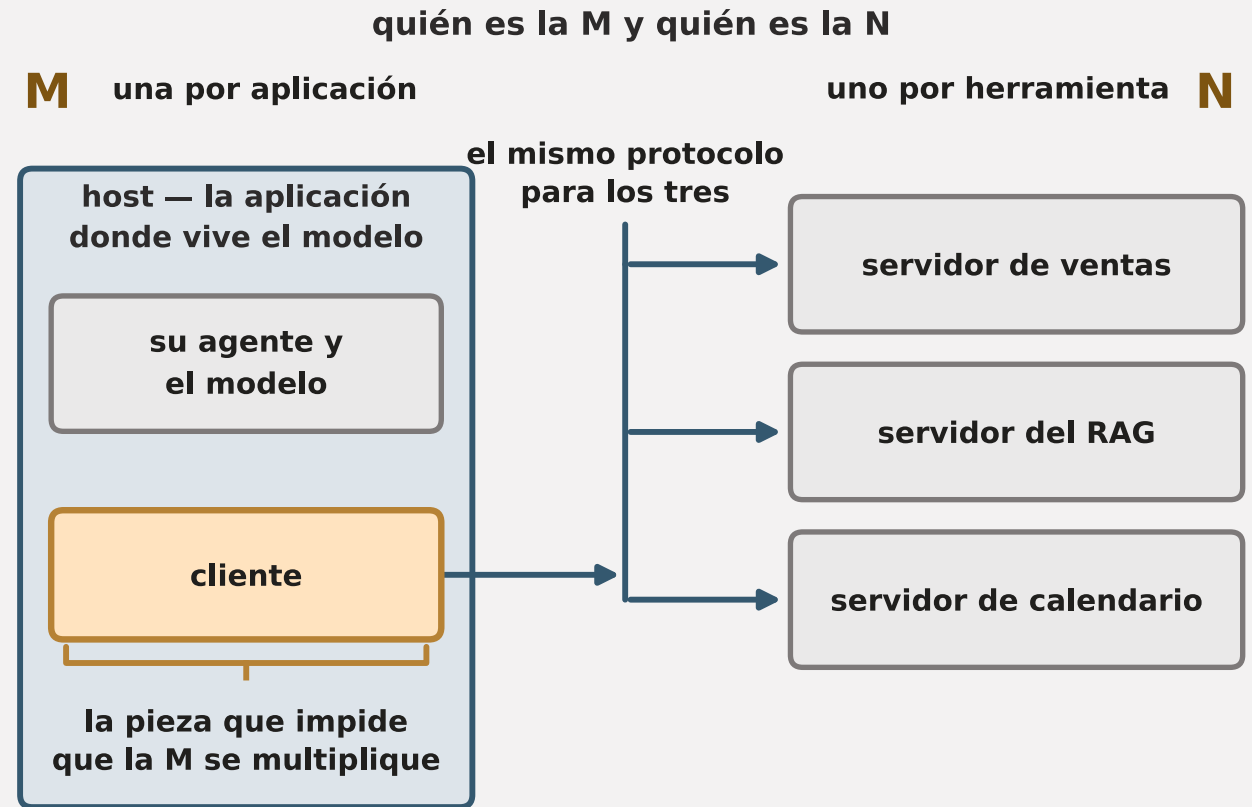
Un protocolo cliente–servidor que estandariza dos cosas

- MCP, Model Context Protocol: estándar abierto propuesto por Anthropic en 2024
- Estandariza cómo una herramienta se PUBLICA y cómo se DESCUBRE. Nada más
- No cambia quién ejecuta, ni cómo el modelo pide, ni quién tiene permiso
- Lo nuevo que hace el cliente: preguntar, en tiempo de ejecución, qué herramientas hay
- «El cliente no sabe de antemano qué herramientas hay»: ésa es la diferencia con el lunes
- Respaldo: seis líneas de las notas del curso y la especificación, con fecha. Cero papers

Fuente: fuentes/notas-teoricas/s3-agentes.md §2.3, líneas 77–79 (estándar abierto de Anthropic 2024; protocolo cliente–servidor; elimina adaptar cada herramienta a cada modelo); curso/actividades/_fuentes/s3-mie.py, líneas 276–277 (el cliente no sabe de antemano). El barrido de fuentes/papers/s3-agentes/ –ReAct, Reflexion, Toolformer y AutoGen– no halló ninguna mención: contenido.md §0

Host, cliente y servidor: quién es la M y quién es la N

- Host: la aplicación donde vive el modelo — su agente, un IDE, un escritorio de chat
- Cliente: el conector que vive dentro del host; normalmente lo trae el framework
- Servidor: lo que expone las herramientas con un contrato estándar
- El host es la M de la cuenta. El servidor es la N
- Y el cliente es la pieza que impide que la M tenga que multiplicarse
- En el cuaderno de hoy las tres son explícitas: ServidorMCP, ClienteMCP y el propio cuaderno



en el cuaderno de hoy el servidor es una clase de Python en el mismo proceso; el transporte real queda fuera del cuaderno

La correspondencia host = M y servidor = N la propone este deck; la tabla de la fuente no la escribe

El catálogo es el esquema de la sesión 11, con otro dueño

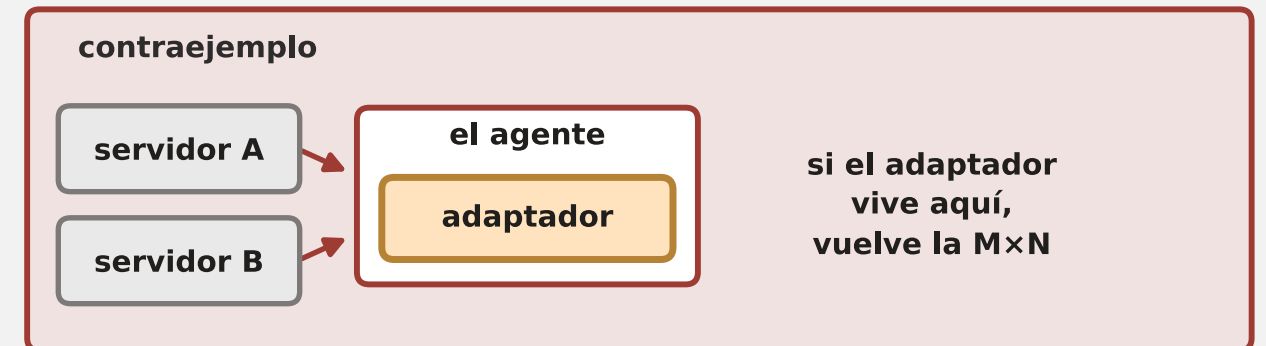
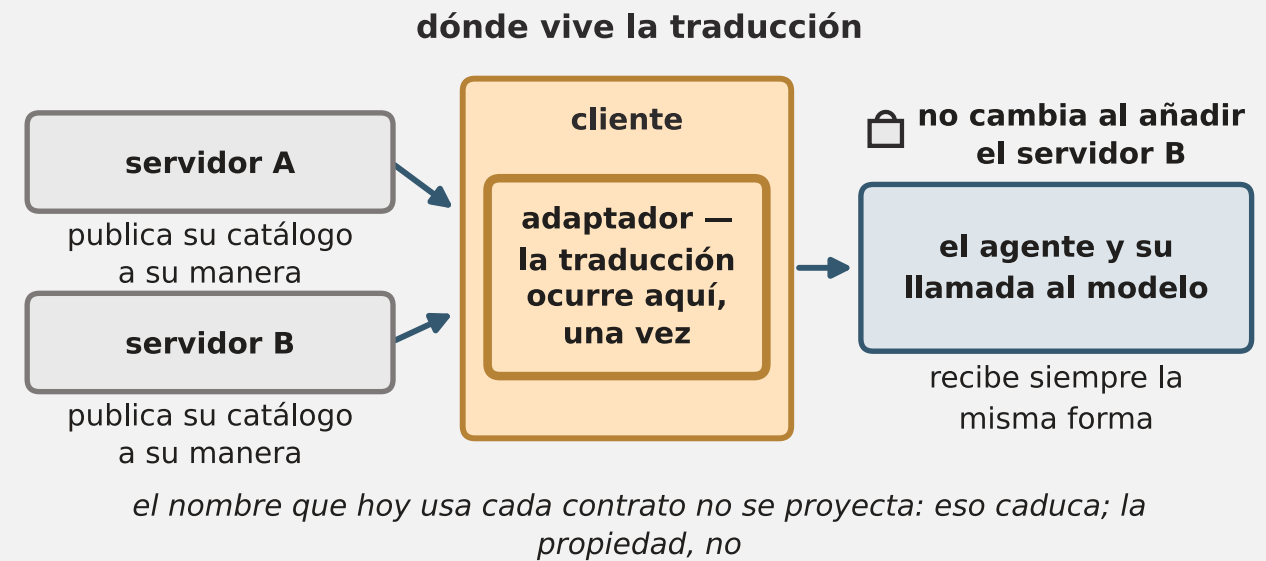
- Es la misma terna del lunes: nombre, descripción y esquema de entrada
- Lo que cambió es quién la escribe y cuándo llega: la escribe quien publica, y llega al ejecutar
- Cambió el dueño del texto, no el texto. La descripción sigue siendo el prompt que decide
- Y viaja como tokens de entrada en CADA iteración del bucle: descubrir no es gratis

```
# lo que publicar() guarda en el catalogo del cuaderno de hoy
{
  "name": "consultar_ventas",
  "description": "Total de ventas de un mes de VENTAS_MES",
  "inputSchema": {
    "type": "object",
    "properties": {"mes": {"type": "string"}},
    "required": ["mes"],
  },
}
# el cliente lo RECIBE: no lo tenia escrito de antemano
```

Fuente: [curso/actividades/_fuentes/s3-mie.py](#), líneas 237–240 (lo que publicar() guarda en el catálogo del simulador del cuaderno) y 368 («más tools descubiertas = más tokens en cada llamada»); [deprecado/slides/sesion-11/outline.yaml](#), d13 (la forma del esquema) y d15 («la descripción es el prompt»). Es el simulador del curso, y sus tres nombres de campo coinciden con los de la especificación: [modelcontextprotocol.io/specification/2026-07-28](#), Tools §«Tool» (consultado 2026-09-30). Segundo testigo, publicado y anterior al protocolo: Alammar & Grootendorst (2024), cap. 7, §«ReAct in LangChain», p. 222 — un framework declara cada herramienta con nombre, descripción y función, e interpola la lista dentro de la plantilla del prompt

El adaptador vive en el cliente, y por eso el agente no se entera

- Dos contratos que no coinciden: el que publica el servidor y el que espera la API del modelo
- La traducción se hace UNA vez, en el cliente. El agente ni la ve ni la mantiene
- Ésa es la razón mecánica por la que un servidor nuevo no cambia una línea del agente
- Si el adaptador viviera en el agente, cada par servidor-modelo volvería a ser una integración
- O sea: sin esta pieza, la cuenta vuelve a ser $M \times N$ aunque haya protocolo
- No proyecto qué nombre usa hoy cada contrato: eso caduca. La propiedad, no



Cinco formas de conectar: quién conoce las herramientas, y cuándo lo sabe

- El mapa de la lectura previa: API directa, function calling, framework, plugin y MCP
- Cableado: las conoce su código, en tiempo de escritura
- Function calling: las conoce el modelo, porque USTEDES se las pusieron en la llamada
- MCP: las conoce el cliente, porque preguntó — en tiempo de EJECUCIÓN
- Framework: las conoce él, porque USTEDES se las pasaron al construir el agente — escritura, otra vez
- Solo «plugin» queda sin describir. El mapa vale por ubicar MCP ENTRE alternativas, no como la única

Fuente: curso/actividades/semana-3/s3-mie-explicacion.html, líneas 92–94 (la lista de las cinco formas, material del profesor); curso/actividades/_fuentes/s3-lun.py, líneas 87–89 y curso/labs/Lab-03-Agentes/tools.py, 91–95 (el cableado); curso/actividades/_fuentes/s3-mie.py, líneas 276–277 (el cliente pregunta). Alammam & Grootendorst (2024), cap. 7, §«ReAct in LangChain», pp. 221–222 (el framework recibe la lista ya construida y la interpola en su plantilla de prompt) y p. 199, Figura 7-1 (el framework como componentes modulares encadenables). Ninguna afirmación del video entra en el deck: contenido.md §5.1

Qué esperar de un servidor MCP según la revisión 2026-07-28

- Revisión vigente: 2026-07-28, consultada el 2026-09-30. La versión es la fecha del último cambio incompatible
- Implementa `server/discover`, obligatorio: qué versiones habla, qué capacidades tiene y quién es
- Publica hasta tres primitivas: `tools`, que invoca el modelo; `resources`, que son datos; y `prompts`, que son plantillas
- `tools/list` devuelve la terna del lunes —`name`, `description`, `inputSchema`— en orden fijo y con `ttlMs` para cachear
- `tools/call` responde `content`; si falla la ejecución, `isError: true`, para que el modelo corrija y reintente
- No guarda sesión: cada petición se basta sola, y el estado entre llamadas viaja como un argumento más

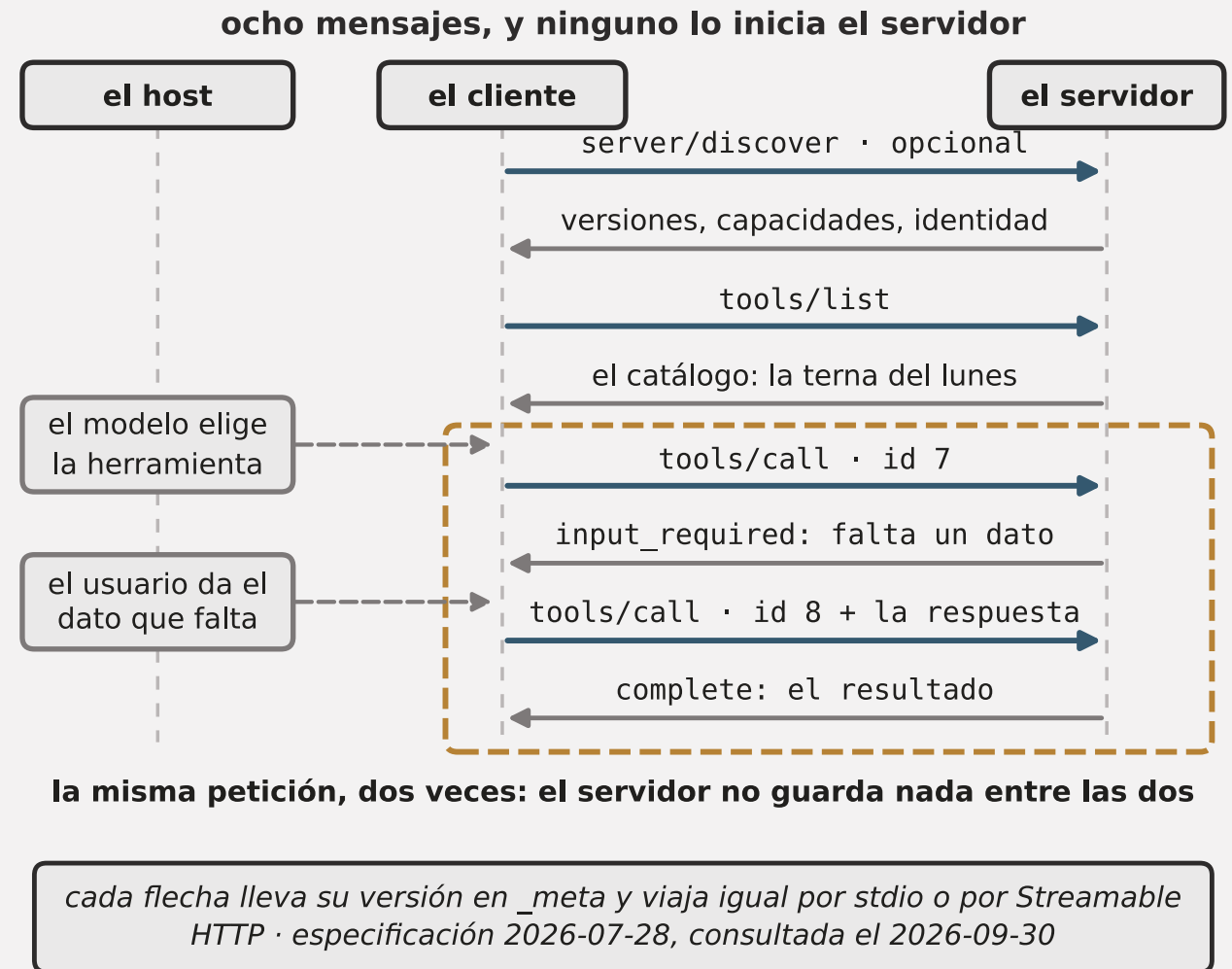
Qué esperar de un cliente MCP: uno por servidor, y se identifica en cada petición

- Lo crea el host, uno por servidor: la relación es 1:1, y el host coordina a todos
- En CADA petición adjunta, en `_meta`, la versión del protocolo, sus capacidades y quién es
- Descubre con `tools/list` y traduce el catálogo a la API del modelo: el adaptador de la d12
- Si la respuesta es `input_required`, reúne lo que falta y REINTENTA la petición con otro `id`

```
{
  "jsonrpc": "2.0", "id": 7, "method": "tools/call",
  "params": {
    "name": "consultar_ventas",
    "arguments": {"mes": "2026-03"},
    "_meta": {
      "io.modelcontextprotocol/protocolVersion": "2026-07-28",
      "io.modelcontextprotocol/clientCapabilities": {}
    }
  }
}
```

Cómo se comunican: JSON-RPC 2.0, y cada petición se basta sola

- Mensajes JSON-RPC 2.0: el cliente pide, el servidor responde. El servidor ya no inicia peticiones
- stdio: el cliente lanza el servidor como subproceso, y va un mensaje por línea
- Streamable HTTP: un solo endpoint, un POST por mensaje; responde JSON o un flujo SSE
- Sin saludo inicial desde esta revisión: la versión viaja en cada petición, no al conectar
- Si al servidor le falta un dato, contesta **input_required** y el cliente reintenta
- El bucle del lunes no cambia: el modelo sigue sin ejecutar nada



La cuenta con sus números — 12 minutos, en grupos

- En su organización: ¿cuántas aplicaciones (M) usarían herramientas de IA? Nómbrénlas
- ¿Cuántas herramientas o sistemas (N) querrían exponer? Nómbrénlos también, no los estimen
- $M \times N = ___$ integraciones a mano. $M + N = ___$ piezas con protocolo
- Comprueben $(M - 1)(N - 1) > 1$ y digan a qué lado cae su caso, con los nombres en la mano
- Y la pregunta que recojo en la puesta en común: ¿qué NO resuelve el protocolo?
- Es el ejercicio de la última celda del cuaderno de hoy: lo hacemos aquí y con casos reales

Las tres cosas que MCP no es

13

«Con MCP el modelo ya puede ejecutar herramientas.» ¿Qué está mal?

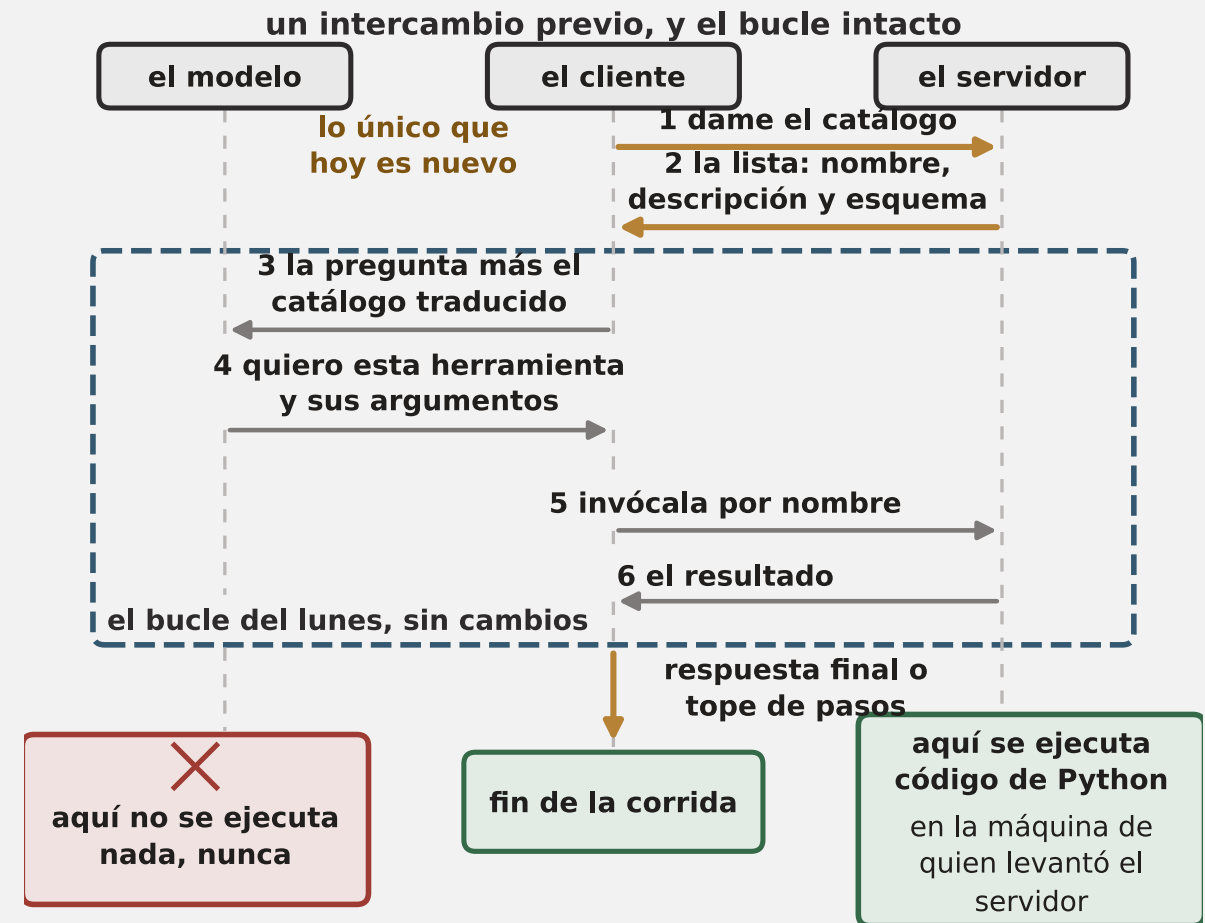
Es el error que la nota docente del material de hoy anticipa por escrito, palabra por palabra

(a) Después de una invocación por nombre, ¿qué pieza ejecuta el código?

(b) ¿En qué máquina corre esa pieza?

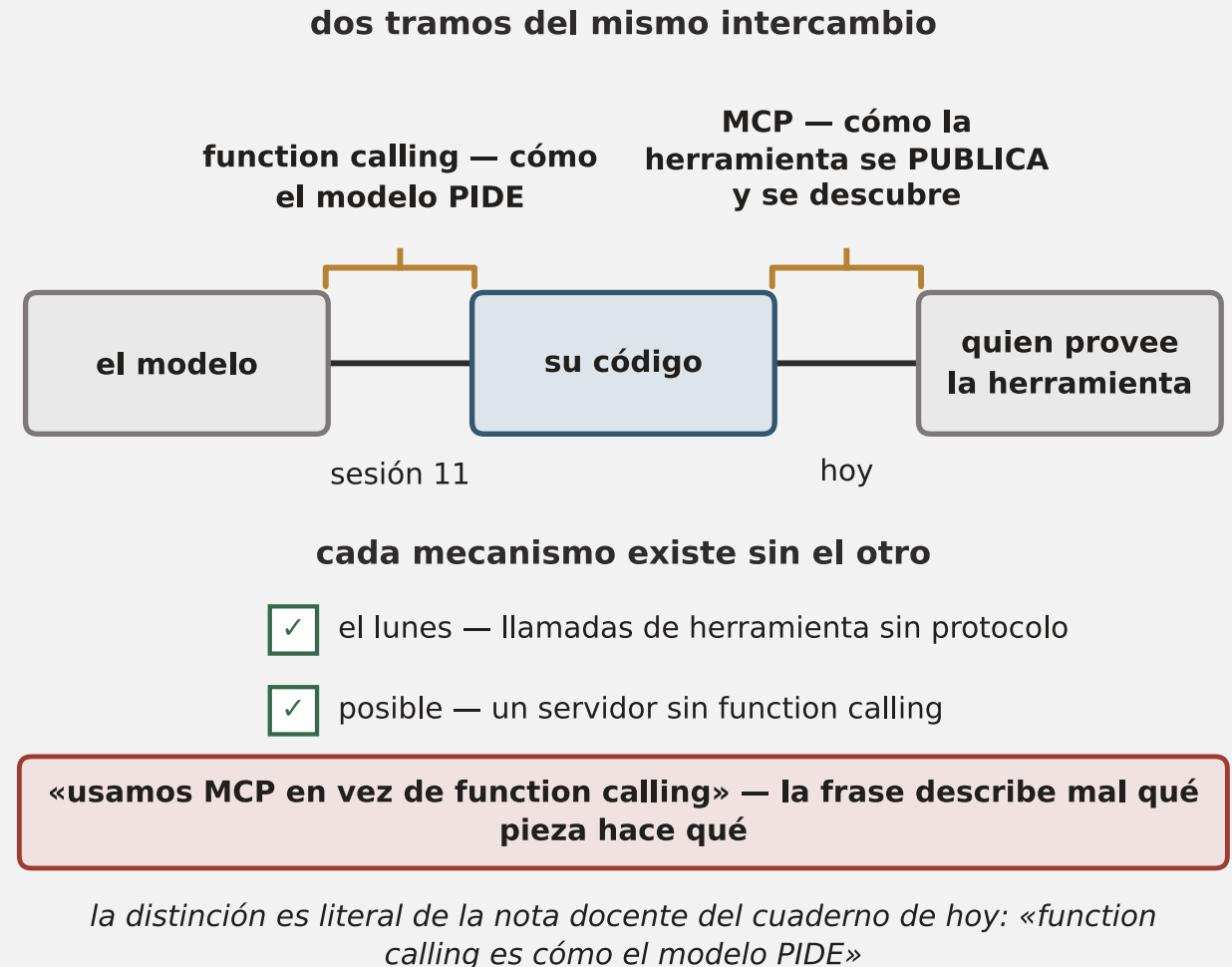
(c) Entonces, ¿qué cambió hoy respecto del lunes? Digan la frase exacta

Pista, y está en el dibujo: en el cuaderno de hoy el servidor es una clase de Python



Function calling y MCP son las dos mitades del mismo intercambio

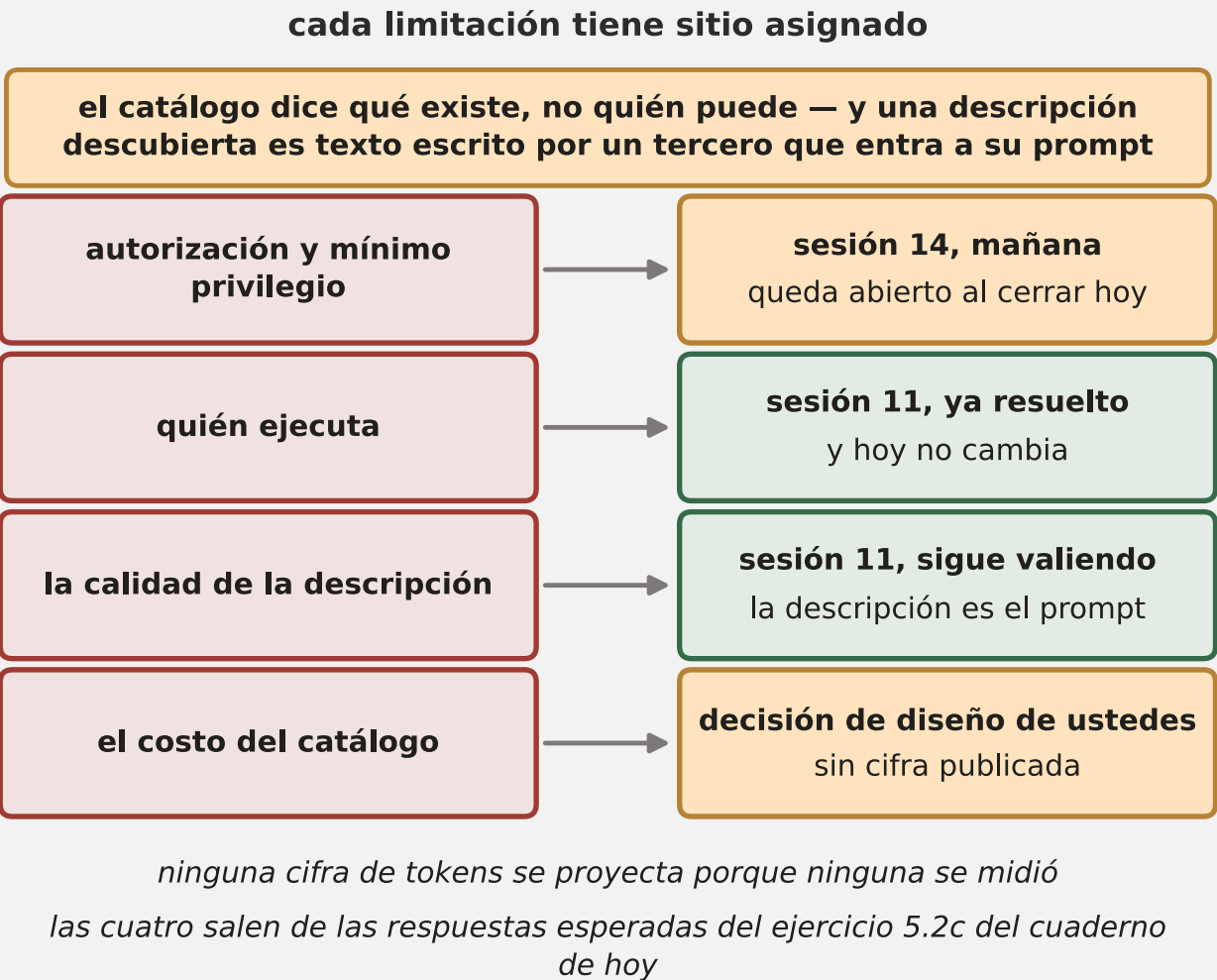
- Function calling estandariza cómo el MODELO PIDE. Entre el modelo y su código
- MCP estandariza cómo la HERRAMIENTA SE PUBLICA y se descubre. Entre su código y quien la provee
- Se puede tener cualquiera de las dos sin la otra: el lunes hubo llamadas y ningún protocolo
- Y un servidor puede servir a un cliente que no use function calling en absoluto
- Decir «usamos MCP en vez de function calling» describe mal qué pieza hace qué
- Es del mismo tipo de error que llamar agente a un chatbot



Fuente: [curso/actividades/_fuentes/s3-mie.py](#), líneas 212–213 y 388–389 (nota docente y cierre del cuaderno: «function calling es cómo el modelo PIDE; MCP es cómo la herramienta se PUBLICA y se descubre»); [deprecado/slides/sesion-11/sesion-11.qmd](#), bloque de function calling «El mecanismo: el modelo emite, tu código ejecuta». La equiparación con «agente/chatbot» se propone como distinción vigilada en [contenido.md](#) §17.3

Lo que MCP no resuelve, y dónde se resuelve cada cosa

- Autorización y mínimo privilegio: descubrir una herramienta no es tener permiso de invocarla — mañana
- Quién ejecuta: sigue siendo su código. Resuelto el lunes, y hoy no cambia
- La calidad de la descripción: el catálogo SON descripciones, y siguen siendo el prompt que decide
- El costo: más herramientas descubiertas, más tokens de entrada en CADA iteración del bucle
- Y la consecuencia que mañana desarrollamos: la descripción descubierta es texto de un tercero
- No proyecto ninguna cifra de tokens: nadie en este repositorio midió cuánto ocupa un catálogo



Fuente: curso/actividades/_fuentes/s3-mie.py, líneas 364–368 (las cuatro cosas que las respuestas del ejercicio 5.2c declaran fuera de alcance) y 356 (el cuaderno remite al «problema de la sesión de mañana»); fuentes/notas-teoricas/s3-agentes.md §5.4, líneas 218–222 (un texto recuperado puede llevar instrucciones que subviertan el comportamiento)

«MCP, hoy estándar de la industria.» ¿Qué le falta a esa frase?

La frase está en el material de esta sesión. La escribió este curso, no un vendedor

Lo que sí proyecto: estándar ABIERTO propuesto por Anthropic en 2024, sin adjetivos de adopción

«Abierto» es una propiedad de la especificación. «De la industria» es una afirmación de adopción

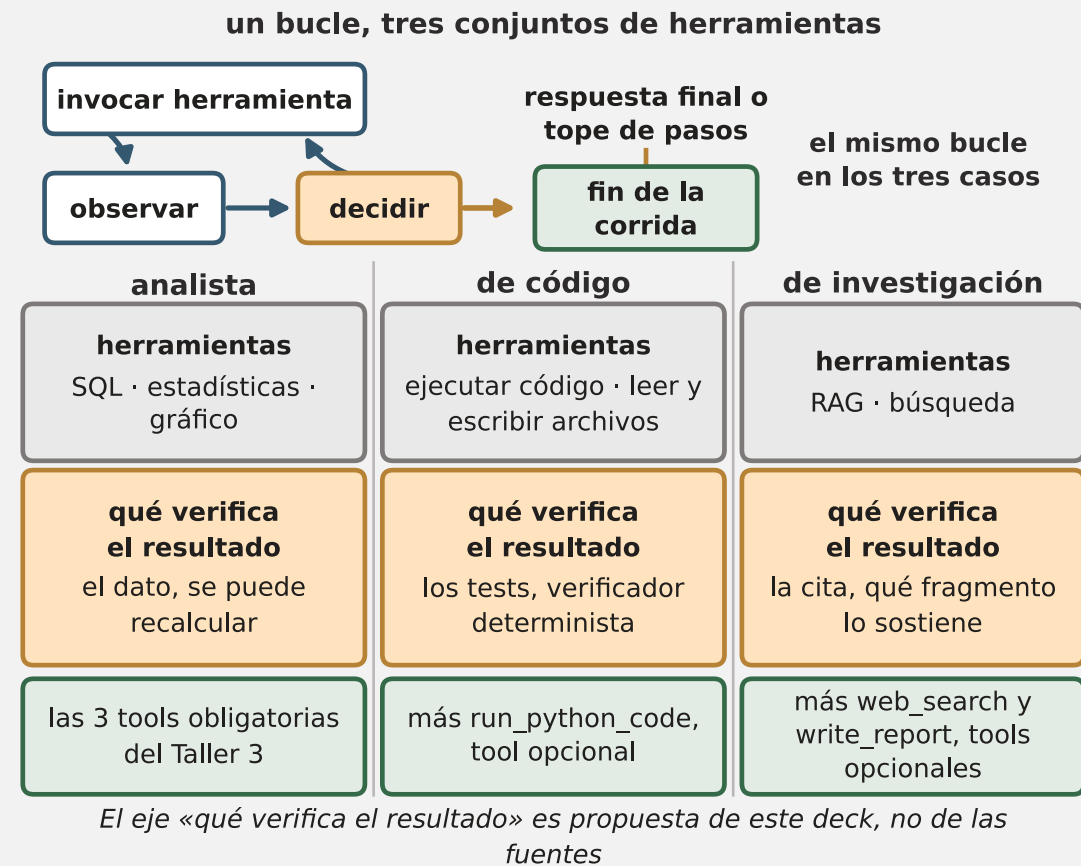
¿Qué tendría que traer para ser citable aquí? Enumérenlo: son cuatro cosas y las saben

Segunda parte: la especificación sí tiene fecha, la revisión 2026-07-28, vigente hoy

¿Esa fecha hace citable la frase? Digan qué prueba una especificación vigente, y qué no

Tres casos de uso, un mismo bucle: ¿qué verifica que la respuesta es correcta?

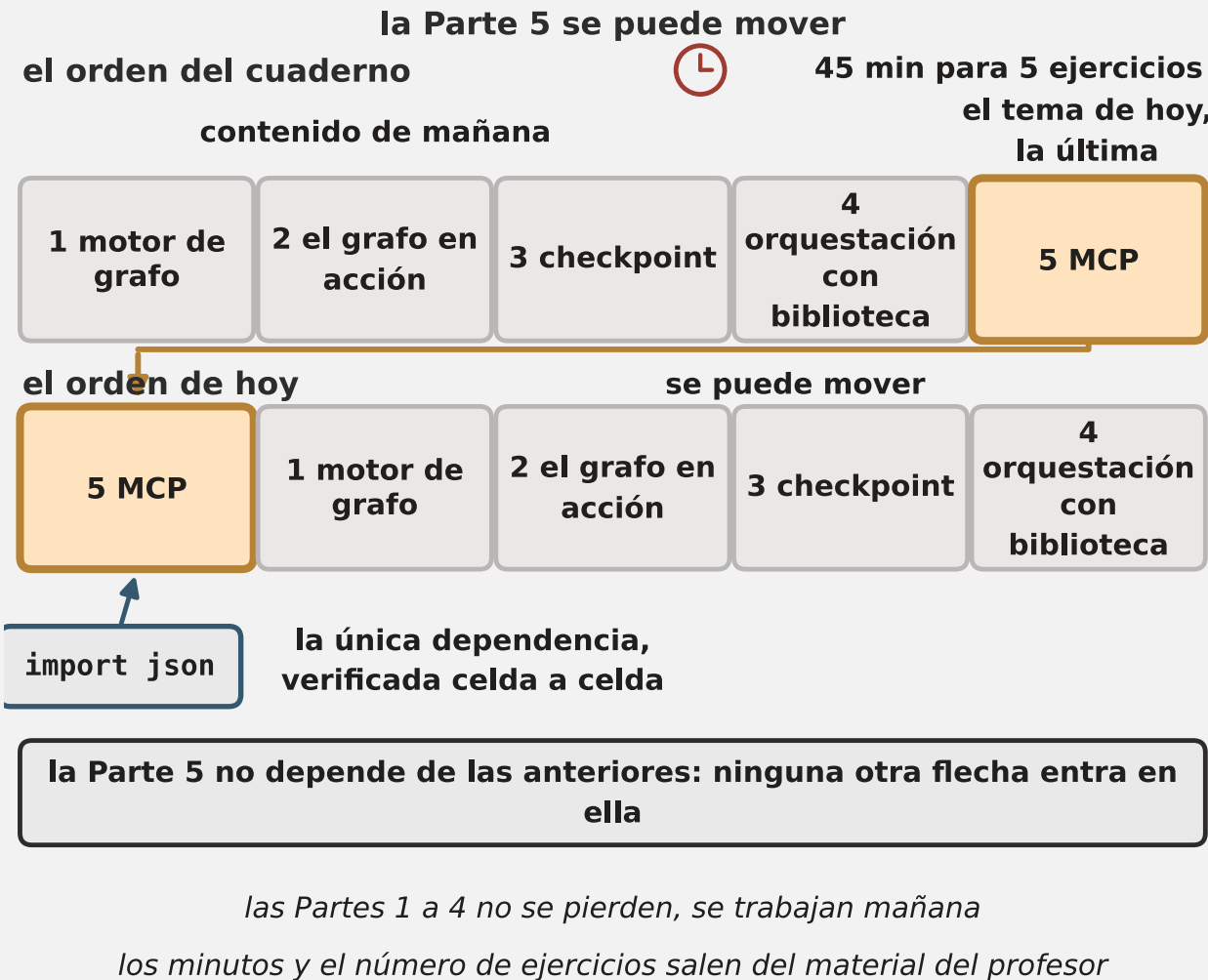
- El eje lo propone el deck; la pregunta, no: el libro la plantea para su propio agente
- Analista: SQL, estadísticas y gráfico. Verifica el DATO: se puede recalcular
- De código: ejecutar código, leer y escribir archivos. Verifican los TESTS, y son deterministas
- De investigación: RAG y búsqueda. Verifica la CITA: qué fragmento sostiene la afirmación
- Los tres son el baseline del sábado más una herramienta: `run_python_code`, o `web_search` y `write_report`
- El del verificador determinista es el único donde Reflexion, de ayer, rinde de verdad



Fuente: el eje organizador es de este deck, contenido.md §6.4. Respaldo parcial del eje: Alammar & Grootendorst (2024), cap. 7, §«ReAct in LangChain», p. 223 (el problema de que nadie juzga el resultado intermedio, y devolver la fuente del dato como remedio). Las tres herramientas opcionales, curso/talleres/taller-03-agentes.md, línea 40. Respaldo celda a celda: curso/talleres/taller-03-agentes.md, líneas 34–38 y 40; curso/labs/Lab-03-Agentes/tools.py, 20–95; fuentes/notas-teoricas/s3-agentes.md, línea 211 (execute_python_code como herramienta peligrosa); _fuentes/s3-mar.py, línea 151; s3-mie-explicacion.html, línea 64

El cuaderno: la Parte 5 primero, y hoy es la única obligatoria

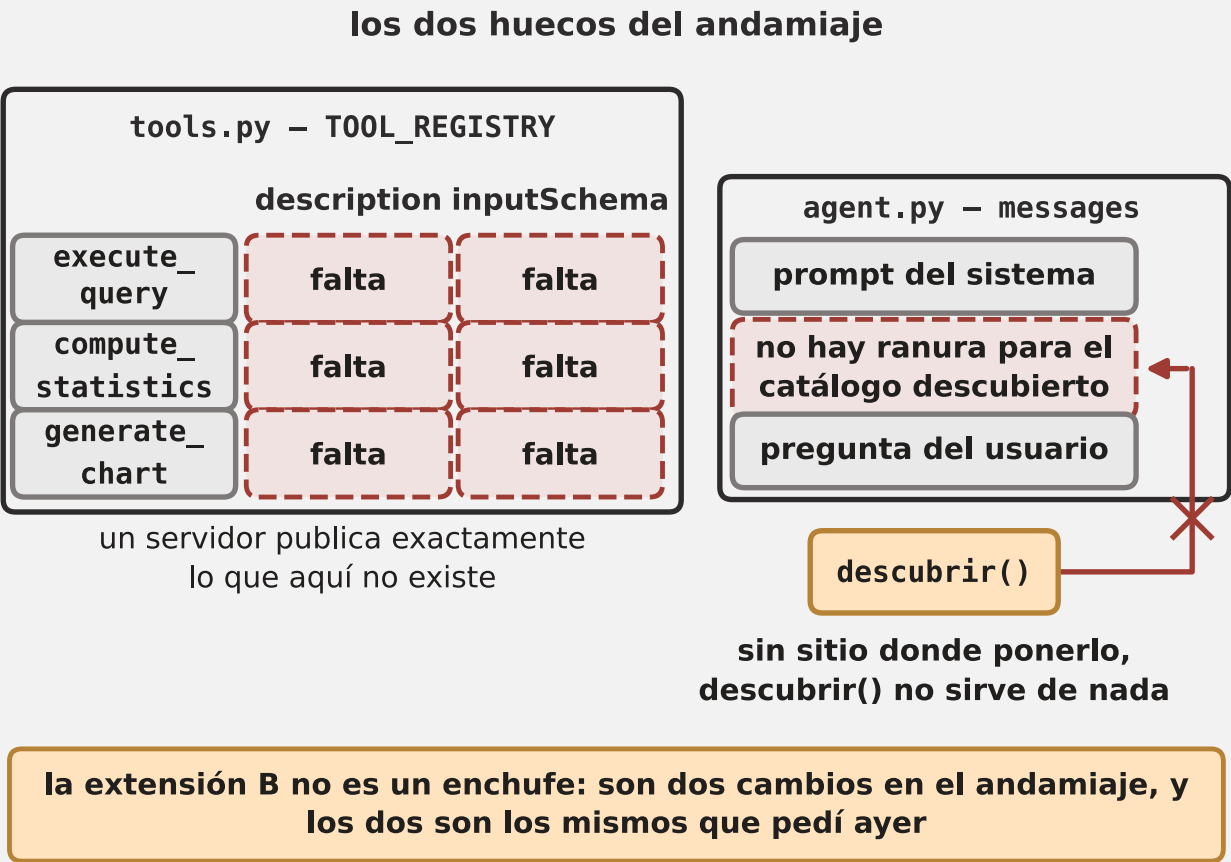
- El cuaderno de hoy tiene cinco partes y cuatro son de mañana: motor de grafo y orquestación
- MCP es la Parte 5, la última. Si el reloj aprieta se cae el tema del día, así que la invierto
- Se puede: la Parte 5 no depende de las anteriores. Su única dependencia es el import json
- 5.1 — escriban el cliente: descubrir() pide el catálogo con tools/list y lo guarda
- 5.2 — levanten un SEGUNDO servidor y comprueben que el agente no cambia ni una línea
- Aviso: el enunciado del 5.2 está incompleto en su cuaderno. Necesitan ClienteMultiMCP, que extiende el 5.1



Fuente: curso/actividades/_fuentes/s3-mie.py, líneas 189–378 (la Parte 5), 276–277 (el 5.1) y 311–348 (el 5.2). Que la Parte 5 solo dependa de cell-0002 está verificado celda a celda en contenido.md §11.1; que el 5.2 no tenga enunciado operativo, en §11.3c. Los nombres tools/list y tools/call se usan como convención del material evaluable: curso/talleres/taller-03-agentes.md, línea 77

Extensión B del Taller 3: qué hay que reescribir del andamiaje

- El taller la pide así: publicar sus tools detrás de un servidor, con catálogo e invocación por nombre
- TOOL_REGISTRY es nombre → función: no guarda descripción ni esquema, que es justo lo que se publica
- Y agent.py no tiene dónde poner el catálogo: messages solo lleva el prompt del sistema y la pregunta
- Sin sitio donde ponerlo, descubrir() no sirve de nada. Es la misma corrección de ayer, cobrada
- El enunciado dice «manifiesto» y el cuaderno dice «catálogo»: es lo mismo. Usaremos catálogo
- Es extensión y no requisito, y con M=1 y N=3 ya saben por qué



el enunciado dice «manifiesto» y el cuaderno dice «catálogo»: es lo mismo
las 3 entradas del registro y los 2 mensajes se leen del laboratorio

Fuente: curso/talleres/taller-03-agentes.md, líneas 75-79 (la extensión B, su criterio de aceptación y la palabra «manifiesto»); curso/labs/Lab-03-Agentes/tools.py, líneas 91-95 y agent.py, líneas 22, 69 y 75-78 (messages, con dos elementos). Los cinco archivos del laboratorio se leyeron completos: contenido.md §12.2 y §12.3

Un protocolo resuelve el descubrimiento; no resuelve la confianza

- El agente descubre en vez de tener cableado: $M+N$ piezas en lugar de $M \times N$ integraciones
- Y solo cuando $(M - 1)(N - 1) > 1$. Con un solo cliente, el argumento es la herramienta $N+1$
- MCP no reemplaza a function calling: uno es cómo el modelo pide, el otro cómo la herramienta se publica
- Ejecutar sigue siendo cosa de su código. Y descubrir no es tener permiso
- Mañana (14): varios agentes, y todo lo que puede salir mal — bucles, costo, herramientas peligrosas
- Sábado: Taller 3, 25 % de la nota final, 3 de octubre. La extensión B sale de hoy y solo de hoy